Article

ubms ai 1: The Banya Number - A Two-Dimensional Number System and LLM Computation without a GPU

Hyukjin Han

Independent Researcher, Hwaseong, Gyeonggi-do, Republic of Korea; bokkamsun@gmail.com

Abstract: Every nonzero number can be written in the form $e^{\ln x}$. Starting from this, the present paper builds an integer-only arithmetic system that runs a large language model (LLM) without multiplication and without a GPU. The paper proposes three things. First, the banya number. A number is two-dimensional by nature. A number that holds magnitude and angle as a single unit is called a banya number. Weights and activations are all banya numbers. Every operation is a unitary rotation, so there is neither conversion nor loss. Second, an expression itself is held as a value. The sum of banya numbers is not collapsed into a real number but preserved as an integer coefficient array over exponent terms of e. Because e is transcendental, equality comparison is exact on the coefficient array alone. Third, differentiation is rotation. The exponential form of e keeps its form under differentiation, and one differentiation is a 90-degree rotation. A fractional derivative is a rotation by an arbitrary angle, the same structure as the fractional-derivative rotation of the Banya Framework. As a result, the entire LLM pipeline runs with additions, shifts, and comparisons only, with no multiplier and no floating-point unit.

Keywords: banya number; two-dimensional number; logarithmic number system; unitary rotation; fractional derivative; integer-only inference; multiplication-free LLM; SIMD; bounded activation; transcendence of e

---

1. Introduction

The system was derived in the following order. Each step was derived by resolving the problem encountered in the previous step.

1. Every number can be written in the form $e^{\ln x}$.
2. If every large-number computation is done by adding exponents, multiplication is unnecessary. This is the same standing as Napier's logarithms of 1614 [1].
3. If the correction values are tabulated, the only cost is the lookup (mapping) speed.
4. If the logarithm is held as an integer, floating point is unnecessary.
5. The goal is an LLM that runs where there is no GPU.
6. The result of an addition is not returned to the logarithm through a table; it is kept unchanged.
7. The value is not converted to a real or an integer; the expression itself is regarded as a new data type.
8. Top-k needs only the order, not the values.
9. Multiplying e by the rotation i is differentiation. Hence softmax is also easy.
10. A number is two-dimensional by nature. Weights are not one-dimensional reals either. This number is called the banya number.
11. Bounded functions such as the sigmoid and tanh are handled without conversion.
12. A banya number fits in one 32-bit union.

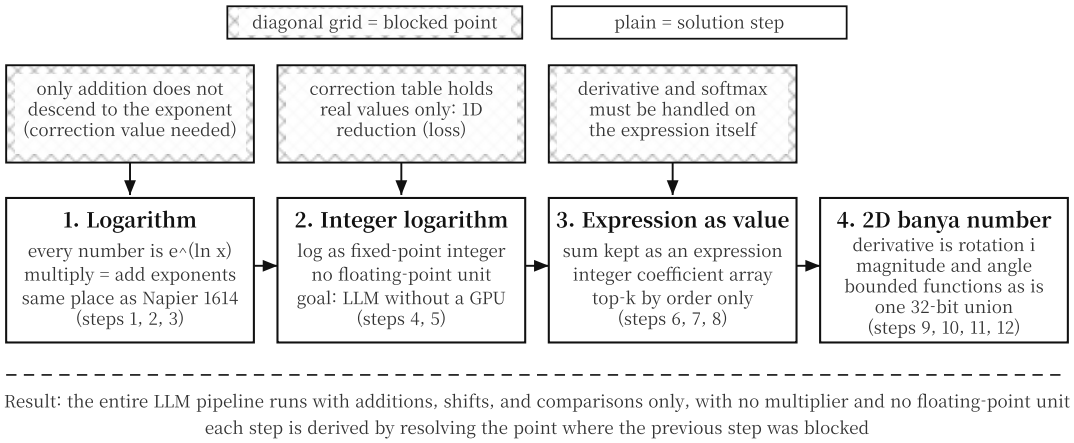


Figure 1. Derivation chain of the system - starting from the logarithm, it proceeds in four steps: integer logarithm, coefficient array that preserves the expression as a value, and the two-dimensional banya number. Each step is derived by resolving the blocked point (diagonal grid) of the previous step.

2. Mathematical Basis

For positive numbers the identity holds directly, because the exponential and the logarithm are inverse functions.

$$x = e^{\ln x} \quad (x > 0) \tag{1}$$

Negative numbers use Euler's formula. Since $e^{i\pi} = -1$, rotating a positive number by 180 degrees gives a negative number. For example, $-5 = e^{\ln 5 + i\pi}$.

$$e^{i\theta} = \cos\theta + i\sin\theta, \qquad -x = e^{\ln x + i\pi} \tag{2}$$

The real part of the exponent fixes the magnitude and the imaginary part fixes the direction. Turning a further 2π gives the same position, so the representations are infinitely many. The implementation handles the angle within one turn only.

Zero is the exception. No power of e yields 0. The implementation reserves the minimum value of the magnitude field (the position inside the structure that holds one value) as 0.

Moving to the exponent lowers every operation by one level.

Table 1. Descent of operations when moved to the exponent.

Original operation	In the exponent
a times b	$\ln a + \ln b$
a divided by b	$\ln a - \ln b$
a to the power n	n times $\ln a$
square root of a	$\ln a$ divided by 2
$a + b$	the larger term plus a correction value

Only addition does not descend. Most of the rest of the present paper describes how this addition is handled.

3. The Banya Number: Two-Dimensional Number and Rotation

The banya number is a two-dimensional number that holds magnitude and angle as a single unit. Weights, activations, and intermediate values are all banya numbers.

$$z = e^{a+i\theta}, \qquad a = \ln|z|, \quad \theta = \text{angle} \tag{3}$$

3.1. One-Dimensional Reduction Is Lossy

Converting to a real number discards the angle. A correction table (LUT) stores real values only, so it is ultimately a one-dimensional reduction and incurs loss. Real differentiation is also a one-dimensional linear transformation and removes the constant term. Hence the constant C is lost on the way back through integration.

3.2. Rotation Is Lossless

Rotation, multiplication by $e^{i\varphi}$, is unitary. It leaves the magnitude unchanged, and rotating back restores the original. Every operation on banya numbers ends with this rotation.

Table 2. Operations on banya numbers and their inverses.

Operation	On banya numbers	Inverse (restores the original value)
Multiplication	add magnitudes and add angles	subtract
Differentiation	rotate by 90 degrees	-90 degrees
Integration	rotate by -90 degrees	90 degrees
Fractional derivative of order α	rotate by α times 90 degrees	the reverse

Differentiating and then integrating returns exactly to the original value.

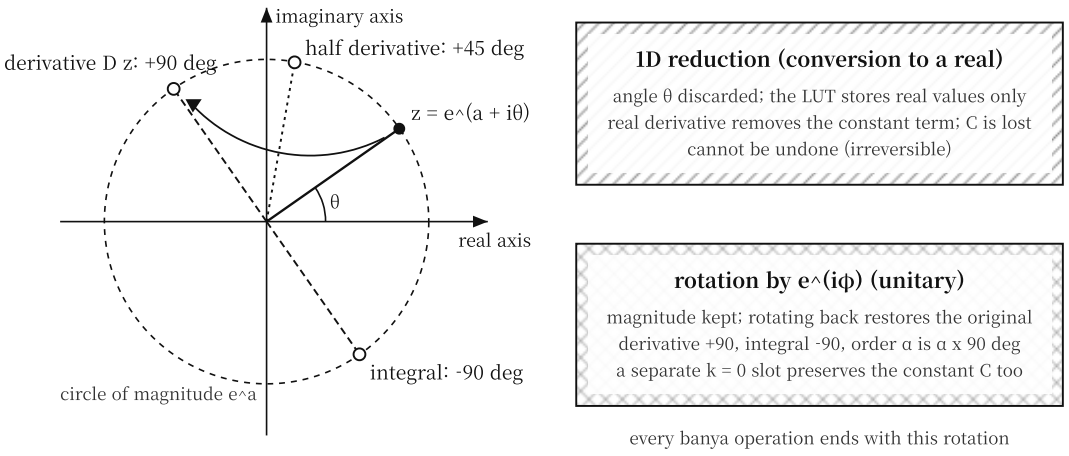


Figure 2. Two-dimensional structure of the banya number and the rotation operations - on the complex plane the magnitude e^a and the angle θ fix one point; differentiation is a $+90$-degree rotation, integration a -90-degree rotation, and the fractional derivative of order α a rotation by $\alpha \times 90$ degrees. The one-dimensional reduction onto the real axis (hatching) loses the angle, while rotation (diagonal grid) preserves the magnitude.

3.3. The Constant C Is Not Lost Either

Differentiation multiplies the term e^{ikx} by ik. Terms with k nonzero survive as a rotation (a turn on the complex plane that keeps the magnitude and changes only the angle). Only the term with $k = 0$, the constant term, is multiplied by 0 and vanishes. This is the identity of the integration constant C. Since the banya number holds its value as a coefficient array, keeping a separate $k = 0$ slot (the position in the coefficient array that corresponds to one exponent) preserves C as well.

3.4. Weights Are Banya Numbers Too

A weight is not a one-dimensional real number. It is a banya number with magnitude and angle. Multiplying by a weight is a scaling and a rotation. The real-valued weights in current use are only the special case where the angle is fixed at 0 or π.

3.5. Fractional Derivative and the Banya Framework

One differentiation is i, and i is 90 degrees. Differentiating α times is i to the power α. A half-order derivative is 45 degrees.

$$D^{\alpha} e^{ix} = i^{\alpha} e^{ix} = e^{i\pi\alpha/2}\, e^{ix} \tag{4}$$

Integer and fractional differentiation merge into a single rotation. This is the same structure as the fractional-derivative rotation of the Banya Framework [11].

4. The Banya Data Type

One banya number fits in one 32-bit union. The same 32 bits are viewed both as raw (the 32-bit original, not split into magnitude and angle) and as the two fields magnitude and angle. The structure is the same as the banya number that holds a two-dimensional number as a single unit.

4.1. Layout

Table 3. Bit layout of the 32-bit union.

Bits	Content	Format
31~16	angle	unsigned 16-bit, 65536 is 2π
15~0	magnitude $\ln \lvert z \rvert$	signed 16-bit, Q8.8 fixed point

x86 and ARM are little-endian, so the magnitude, declared first, occupies the lower half and the angle the upper half. Because the angle sits in the upper half, when the raw values are added the bits that overflow (the upper part of an addition result that exceeds the bit width and is cut off) from the angle are discarded outside the 32 bits. The hardware performs the 2π wrap-around (counting again from 0 once the angle passes a full turn of 360 degrees) at no extra cost. The whole structure is 4 bytes, so it is loaded into one register when passed as a function argument.

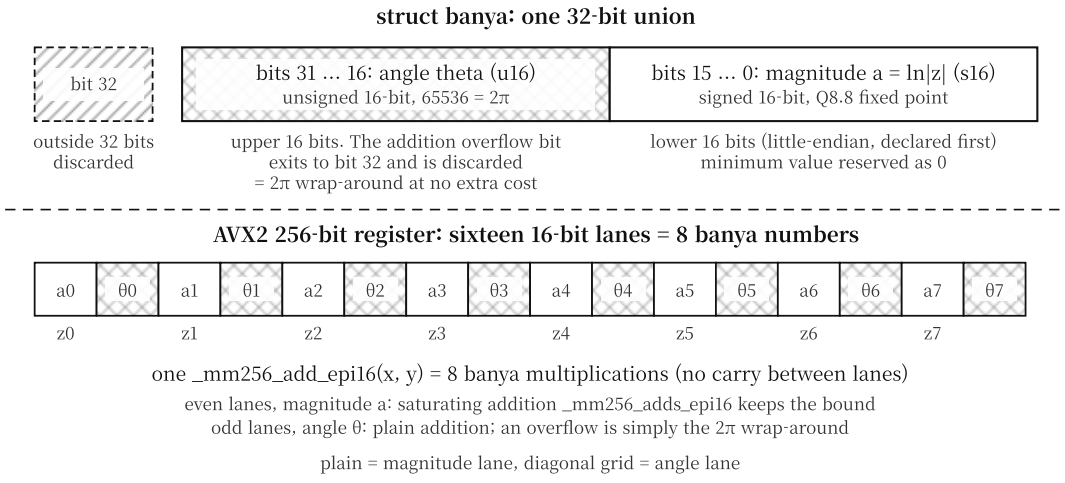


Figure 3. Bit layout of the 32-bit union and SIMD loading - the overflow bit of the angle in the upper 16 bits is discarded outside the 32 bits and becomes the 2π wrap-around, and eight banya numbers are loaded into the sixteen 16-bit lanes of an AVX2 256-bit register as alternating magnitude and angle lanes, so that one instruction performs eight multiplications.

4.2. Definition

```
typedef unsigned int   u32;
typedef short          s16;
typedef unsigned short u16;

struct banya {
    union {
        u32 raw;            // raw 32 bits, used internally only
        struct {
            s16 a;          // magnitude ln|z|, Q8.8
            u16 theta;      // angle, 65536 = 2pi
        } p;
    };

    // multiplication: add magnitudes, add angles
    banya operator*(const banya& y) const {
        banya r;
        r.p.a     = p.a + y.p.a;
        r.p.theta = p.theta + y.p.theta;
        return r;
    }

    // division: subtract magnitudes, subtract angles
    banya operator/(const banya& y) const {
        banya r;
        r.p.a     = p.a - y.p.a;
        r.p.theta = p.theta - y.p.theta;
        return r;
    }

    // derivative 90 degrees, integral -90 degrees
    banya deriv() const { banya r = *this; r.p.theta += 16384; return r; }
    banya integ() const { banya r = *this; r.p.theta -= 16384; return r; }

    // fractional derivative: alpha_q is alpha as an integer in 1/16384 units
    banya frac_deriv(int alpha_q) const {
        banya r = *this; r.p.theta += (u16)alpha_q; return r;
    }

    // equality: one raw comparison
    bool operator==(const banya& y) const { return raw == y.raw; }

    // no integer conversion operator, on purpose
};
```

Reading the 32 bits as an integer gives a meaningless number in which angle and magnitude are mixed. The banya number is not a variant of the integer but a number system standing on its own, so integer conversion is blocked. Code that uses a banya number as an integer is then rejected at compile time. The raw view is used only when loading registers in bulk operations.

4.3. Batch Processing with SIMD

In x86 SIMD, the addition of 16-bit lanes (the equal-sized sections into which a SIMD register is divided) does not carry (the 1 that moves up to the next digit in addition) between lanes. The

magnitude field and the angle field of the banya number fit 16-bit lanes directly. One instruction performs several banya multiplications at once.

Table 4. Banya multiplications per SIMD instruction.

Instruction	Register	Banya multiplications per instruction
MMX paddw (1997)	64-bit	2
SSE2 _mm_add_epi16	128-bit	4
AVX2 _mm256_add_epi16	256-bit	8
AVX-512 _mm512_add_epi16	512-bit	16

```
#include <immintrin.h>

// multiply 8 banya numbers by 8 banya numbers in one instruction
// even lanes: magnitude, odd lanes: angle
__m256i banya_mul8(__m256i x, __m256i y)
{
    return _mm256_add_epi16(x, y);
}
```

The angle lanes use plain addition, so an overflow is simply the wrap-around. If the magnitude lanes use saturating addition, an overflow stops at the maximum (`_mm256_adds_epi16`). The bounded handling of Section 8 enters in one instruction. The two instructions are blended with a lane mask so that the magnitude saturates and the angle wraps around.

4.4. Comparison with Existing Data Types

Table 5. Comparison of existing complex data types and banya.

Data type	Storage	One multiplication
C99 _Complex, C++ std::complex	real part and imaginary part	4 real multiplications, 2 additions
banya	log magnitude and angle	two 16-bit integer additions
banya + AVX2	pack of 8 (eight carried together in one register)	8 in one instruction

There is no existing data type in C or C++ that holds magnitude and angle. banya fills that gap.

5. Coefficient Representation: Holding the Expression as a Value

Adding two banya numbers does not merge them into one magnitude and one angle. Converting to a real number here reduces to one dimension and incurs loss. Hence the sum is held not as a merged value but as the expression itself.

5.1. Definition

Take the exponent step (the smallest interval that divides the exponent) as $1/256$ and let $q = e^{1/256}$. A value is then written as follows.

$$v = \sum_k c_k \, q^k, \qquad q = e^{1/256}, \quad c_k \text{ integers} \tag{5}$$

A value is an array of integer coefficients, one attached to each exponent slot.

```
struct banya_sum {
```

```
    int  lo;   // lowest exponent slot number
    int  n;    // number of slots
    int* c;    // integer coefficient per slot, value = sum c[i] * e^((lo+i)/256)
};
```

Table 6. Operations on the coefficient array and their error.

Operation	Handling	Error
Addition, subtraction	add coefficients of the same slot	none
Multiplication	polynomial product of the coefficient arrays; exponents add	none
Division by one term	shift the slots	none
Equality comparison	compare the coefficient arrays	none
Magnitude comparison	check from the largest slot	result exact
Normalization	truncate the small slots	only as much as set

5.2. Equality Comparison Is Exact

$q = e^{1/256}$ is transcendental. This follows from the Lindemann-Weierstrass theorem: for a nonzero algebraic number a, e^a is transcendental. Therefore a nonzero polynomial with integer coefficients cannot vanish at q. If the coefficient arrays differ, the values differ without exception. The floating-point case in which $0.1 + 0.2$ fails to equal 0.3 does not occur here.

With base 2 this breaks. Since $2^1 = 2^0 + 2^0$, different arrays give the same value. This is why the base is taken as e.

5.3. Magnitude Comparison

Magnitude comparison must look at the values. Differences are accumulated from the largest exponent slot downward, and the comparison stops once the gap is so wide that the remaining slots cannot overturn it even when all are added. Most comparisons end within the first few slots.

5.4. Normalization

As multiplications continue, the slots increase. At the end of each layer, the slots whose values are very small are truncated. Error arises only here, and only as much as is set. Together with the rounding of an external real number to the quantization step (the interval when a real number is rounded to values at fixed spacing) on entry, there are only two places where error arises.

5.5. Bucket Inner Product

If the weights are 4-bit logarithms, there are only 8 exponent slots. Grouping the inner product by slot gives the following.

$$\sum_i a_i \, e^{k_i} = \sum_k e^k \Big(\sum_{i:\, k_i = k} a_i \Big) \tag{6}$$

Adding the activations into 8 slot buckets (bins that collect and add the terms with the same exponent) yields the inner product as an exact banya number. An inner product of length 4096 ends with 4096 additions and the collection of 8 slots. There is not a single multiplication.

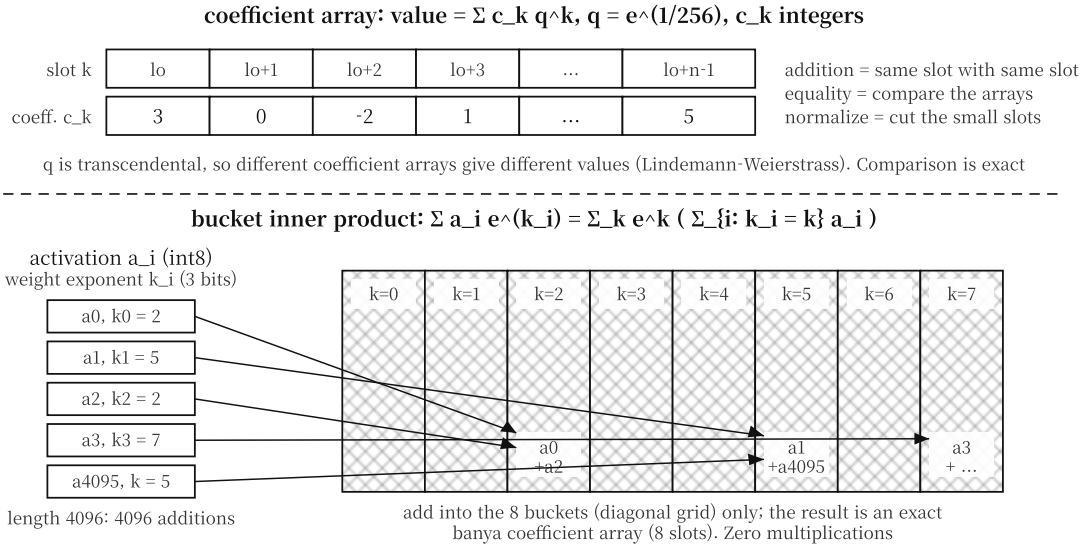


Figure 4. Coefficient array and bucket inner product - a value is an array of integer coefficients attached to exponent slots (top), and an inner product of length n comes out as an exact banya coefficient array by only adding the activations into the 8 buckets (diagonal grid) indexed by the weight exponent (bottom). There is not a single multiplication.

6. Mixed Structure and Integer Logarithm

Existing logarithmic number systems (LNS) use a correction table to perform addition inside the logarithm. The present paper performs addition outside the logarithm. Then no correction table is needed.

6.1. Structure

A weight is 1 sign bit and 3 exponent bits, 4 bits in total. Its value is plus or minus 2 to the power k. Activations stay int8. A multiplication is one shift, and accumulation is int32 addition. The result is passed to the next layer as an ordinary integer.

```
// w: 4-bit log weight (upper 1 bit sign, lower 3 bits exponent)
// a: int8 activation
int acc = 0;
for (int i = 0; i < n; ++i) {
    unsigned char w = W[i];
    int p = (int)A[i] << (w & 7);     // shift instead of multiplication
    acc += (w & 8) ? -p : p;          // sign
}
```

6.2. Conversion Cost

Going from the logarithm to an ordinary number is a shift. Going from an ordinary number to the logarithm is one CLZ instruction. Both x86 and ARM do it in 1 clock. If more fractional bits are needed, the bits below the most significant bit are used directly (Mitchell's approximation, 1962 [2]).

```
int lg = 31 - __builtin_clz(x);     // floor(log2 x)
```

6.3. Range of the Integer Logarithm

Holding the logarithm as a fixed-point integer gives a very wide range. The error is constant as a ratio. Large and small numbers have the same error ratio.

Table 7. Range and step size of integer logarithm formats.

Format	Range	Step size
int32 Q8.8	$2^{\pm 8\,000\,000}$	0.27%
int32 Q16.16	$2^{\pm 30\,000}$	0.001%
int64 Q32.32	$2^{\pm 2\times 10^9}$	on the order of 10^{-10}%

6.4. Size of a Correction Table, if One Were Used

Addition inside the logarithm requires the correction value $f(d)$. Its size is recorded here for comparison.

$$f(d) = \log_2\left(1 + 2^{-d}\right) \tag{7}$$

With F fractional bits, the number of table entries (the individual values stored in the table) is about $(F + 1.5)$ times 2 to the power F.

Table 8. Size of the correction table.

Format	Stored per entry	Linear interpolation
Q8.8	about 2,400 entries, 5 KB	not needed
Q16.16	about 1.15 million entries, 4.6 MB	about 4,500 entries, 18 KB
Q32.32	impossible	about 550 thousand entries, 4.4 MB

The banya number does not use this table. The table holds real values only, which is a one-dimensional reduction.

6.5. float Is Already Half Logarithmic

Reading the bits of an IEEE 754 float as an integer gives roughly $\log_2(x)$. Hence one integer addition gives an approximate multiplication.

```
int ia = *(int*)&a;
int ib = *(int*)&b;
int ic = ia + ib - 0x3F800000;    // approximate a*b with integer addition only
float c = *(float*)&ic;
```

7. Application to LLMs

LLM inference runs on the CPU from beginning to end. There is not a single operation that requires a GPU. The GPU is merely faster. The heavy parts are two: long-context attention and long-prompt prefill.

7.1. Handling by Stage

Table 9. Operations and handling by stage of LLM inference.

Stage	Operation	Handling	Cost
Q, K, V construction, output projection, FFN	weight and activation	shift or bucket inner product	light
Accumulation	addition	int32 addition	light
Attention Q with K, its result with V	activation with activation	to the logarithm by CLZ, then shift	proportional to context length
Softmax	exponential	accumulated value placed directly into the exponent slot	light
Output top-k	order only	integer comparison	light

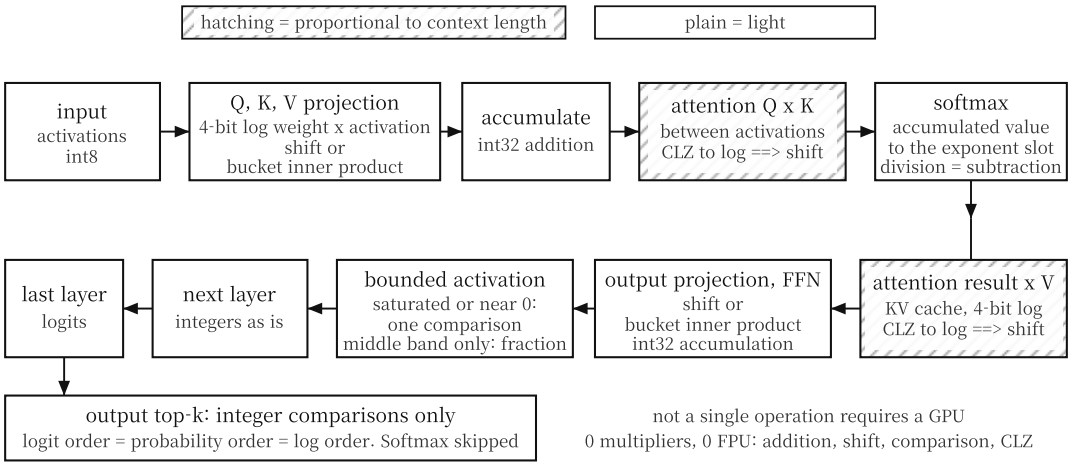


Figure 5. Integer-only path of one LLM inference layer - the product of weight and activation is a shift or a bucket inner product, accumulation is int32 addition, the attention product between activations goes to the logarithm by CLZ and then shifts, softmax places the accumulated value into the exponent slot, and the output ends with integer comparison. The only stages proportional to the context length (hatching) are the two attention products.

7.2. Top-k Ends with Comparison

Both e^x and log are increasing functions. Hence the order of the logits is the order of the probabilities and the order of the log values. If only the single largest is selected, softmax is not executed. Even when sampling by probability, only the top 40~50 entries are converted to probabilities, not the tens of thousands in the vocabulary.

7.3. Softmax and Differentiation

The exponential form of e keeps its form under differentiation.

$$\frac{d}{dx}e^x = e^x, \qquad \frac{d}{dx}e^{kx} = ke^{kx}, \qquad \frac{d}{dx}e^{ix} = ie^{ix} \tag{8}$$

In the forward direction, $\log p_i = x_i - \log(\text{total sum})$, so the division is one subtraction. In the backward direction (training), the gradient is $p_i(\delta_{ij} - p_j)$, which uses only the p already obtained. No new exponential needs to be computed.

7.4. KV Cache and Context Length

Storing the KV cache as 4-bit logarithms as well reduces memory to below half, and the attention multiplications become shifts too. With a GQA model, in which several heads share K and V, the cache decreases further. A sliding window that looks only at a fixed recent length blocks the growth with context length. With a small 1~3B model, prefill is short as well.

8. Bounded Activation Functions

The sigmoid and tanh are also handled inside the banya number without conversion to real numbers.

8.1. The Accumulated Value Is Placed into the Exponent Slot

In the mixed structure, the layer accumulator x is an ordinary integer. Placing it directly into the exponent slot of a banya number gives e^x. The number of operations is 0.

$$\sigma(x) = \frac{e^0}{e^0 + e^{-x}}, \qquad \tanh(x) = \frac{e^0 - e^{-2x}}{e^0 + e^{-2x}} \tag{9}$$

The numerator and the denominator are two-term coefficient arrays. The fraction is kept as a fraction. $2x$ is one shift.

8.2. Saturation Takes One Comparison

Once the input grows even slightly, the output converges to the saturation value (the terminal value, 1 or -1, that no longer changes however large the input becomes). Looking only at the integer part of x decides most cases.

Table 10. Handling of bounded functions by input interval.

Interval	tanh	σ	Cost
x greater than 8	exactly 1	exactly 1	one comparison
x near 0	x unchanged	$1/2 + x/4$	one shift
x less than -8	exactly -1	exactly 0	one comparison
middle band	fraction	fraction	two banya terms

In Q8.8, once x exceeds 8, tanh is exactly 1 within Q8.8 precision. This is not an approximation but the true value. Computation is needed only in the middle band (the input interval that is neither saturated nor near 0).

```
// x: accumulated integer (Q8.8), SAT = 8 * 256
if (x >  SAT) return BANYA_ONE;                  // saturated 1
if (x < -SAT) return BANYA_NEG_ONE;              // saturated -1
if (x > -LIN && x < LIN) return x;               // near 0: tanh(x) = x
return banya_frac(e0 - e(-2*x), e0 + e(-2*x));   // middle band only, as a fraction
```

8.3. The Bound Is a Projection of Rotation

Hyperbolic functions and circular functions are related by a 90-degree rotation. Inserting the Gudermannian function $\mathrm{gd}(x)$, tanh is the projection (dropping a two-dimensional point perpendicularly onto the real axis and keeping only that coordinate) onto the real axis of an angle on the unit circle.

$$\tanh(x) = -i\tan(ix), \qquad \tanh(x) = \sin(\mathrm{gd}(x)), \qquad \mathrm{gd}(x) = 2\arctan(\tanh(x/2)) \tag{10}$$

tanh is confined between -1 and 1 because the radius of the unit circle is 1. Hence the tanh output is stored as a banya number of magnitude 1 and angle $\mathrm{gd}(x)$. Since the magnitude is 1, the data type keeps the bound automatically. The angle is 16 bits, so an overflow wraps around automatically. Moving the real exponent (hyperbolic) to the imaginary exponent (rotation) is the mathematical form of what it means to keep two dimensions intact.

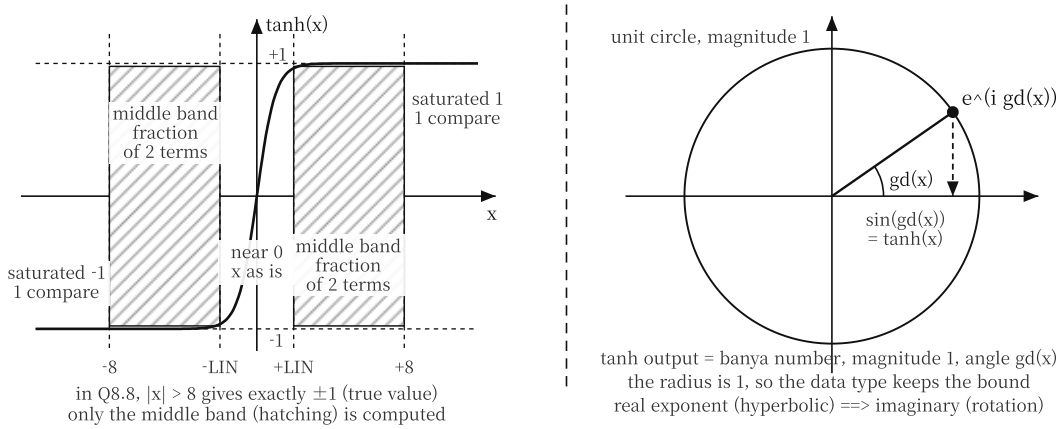


Figure 6. Three intervals of a bounded activation function and the rotation projection - on the left, the saturation intervals of $\tanh(x)$, the linear interval near 0, and the middle band (hatching), of which only the middle band needs the fraction computation. On the right, projecting the angle $\mathrm{gd}(x)$ on the unit circle onto the real axis gives $\tanh(x)$; the magnitude is 1, so the data type keeps the bound.

8.4. Classifying the Activations

When multiplying by the weights of the next layer, the activations are divided into three paths.

1. Saturation values 1 and -1: the weight is used unchanged. Zero multiplications; only the sign is attached.
2. Values near 0: they take the shift path as integers.
3. Middle-band values: they are held as fractional banya numbers and placed into the bucket inner product.

Only the third path spends resources on conversion. Its proportion is measured from the activation distribution in the engine.

8.5. Activation Functions Native to the Banya Number

ReLU costs 0, since it only looks at the sign bit. An angle clamp can be used instead of tanh. The 16-bit angle is cut at plus or minus 90 degrees. This gives a bounded activation function with a single integer and no Gudermannian computation.

```
unsigned short angle_clamp(short theta) {       // 90 degrees = 16384
    if (theta >  16384) theta =  16384;
    if (theta < -16384) theta = -16384;
    return (unsigned short)theta;
}
```

9. Prior Work and Differences

Work that removes neural-network multiplication by logarithms exists [5,6]. Most of it is on the dedicated-chip side [7,8,10]. There is no case in which a number system that treats two-dimensional numbers by rotation and holds the expression itself as a value has been used for an LLM.

Table 11. Prior work and products, and how the present paper differs.

Work or product	Year	Content	Difference from the present paper
Napier's log tables, slide rule [1]	1614	multiplication as log addition	the same starting point
Mitchell's approximation [2]	1962	log2 approximated from bits	borrowed for conversion
Clenshaw and Olver's level-index arithmetic [3]	1984	numbers represented by iterated powers of e	no two dimensions, no rotation, no LLM
Exact real arithmetic		expression preserved as value	general-purpose computation; speed not the goal
LNS-Madam [5]	2021	NVIDIA and Caltech, low-precision training in LNS	GPU-class hardware
PNNL LLM-LNS [7]	2024	LNS matched to LLM outliers	hardware co-design
QAA-LNS [8]	2025	log addition approximated by line segments, MAC area up to 32.5%, energy up to 53.5% saved	the present paper does no log addition
Tensordyne (formerly Recogni) Napier [9]	2025	logarithmic inference chip, claims 17x tokens per watt versus NVIDIA	dedicated chip; the present paper uses a plain CPU
llama.cpp [12]	2023	4-bit linear quantization on CPU	linear spacing, uses multiplication
Microsoft BitNet b1.58 [13]	2024	multiplication removed with weights -1, 0, 1	3 values; the present paper has 16 log values
Microsoft T-MAC [14]	2024	low-bit multiplication by table lookup	linear weights, one-dimensional

The differences are four.

1. Numbers are two-dimensional banya numbers and operations are rotations. All existing work operates on one-dimensional real numbers.
2. Log addition is avoided, not approximated. Accumulation is outside the logarithm, and sums stay as expressions.
3. It is software on a plain CPU. The integer-only logarithmic LLM engine is an unoccupied position.
4. The transcendence of e makes equality comparison exact, and inference, training, and fractional differentiation are handled in one data type.

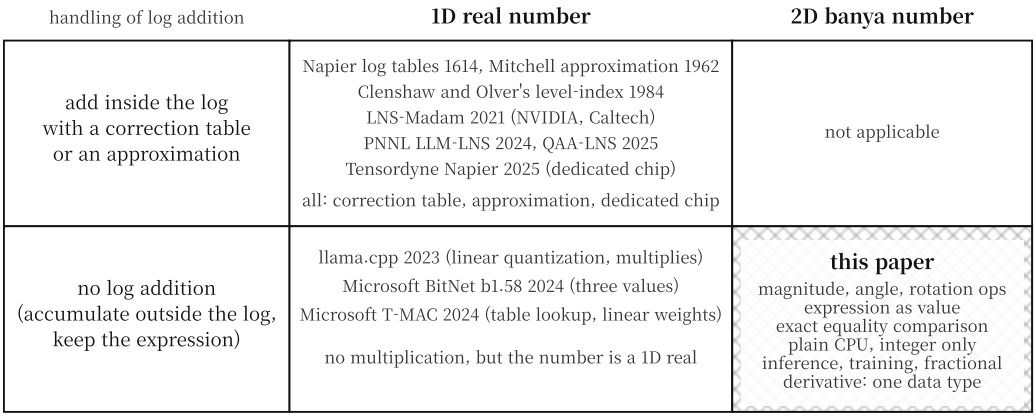


Figure 7. Position map of prior work - the horizontal axis is the dimension of the number (one-dimensional real, two-dimensional banya number), and the vertical axis is the handling of log addition (correction table or approximation, accumulation outside the logarithm with the expression preserved). All existing work and products sit in the one-dimensional real cells, and only the present paper sits in the cell (diagonal grid) that is two-dimensional and avoids log addition.

10. Cost Estimate

Looking at the arithmetic units alone, energy decreases by 2~4 times; with the bit width also reduced, the target for the whole system is several-fold up to roughly tenfold.

10.1. Energy per Operation (Horowitz 2014, 45 nm)

Table 12. Energy per operation [4].

Operation	Add pJ	Multiply pJ	Multiply over add
8-bit integer	0.03	0.2	about 7 times
32-bit integer	0.1	3.1	about 30 times
FP16	0.4	1.1	about 3 times
FP32	0.9	3.7	about 4 times

Reading one 32-bit word from DRAM costs about 640 pJ. That is 170 times an FP32 multiplication. When an LLM generates tokens one at a time, the bottleneck is memory, not arithmetic. Hence reducing the weight bit width is the largest gain.

10.2. CPU Inference Speed (7B Model)

With 4-bit weights the model is about 3.5 GB. Ordinary dual-channel DDR5 delivers about 80 GB per second, so the upper bound on token generation is around 20 per second. Prefill is about 14 GFLOP per token, so a 1,000-token prompt takes several seconds to tens of seconds on a CPU.

10.3. Share of Attention (7B, 32 Layers, Dimension 4096)

Table 13. Share of attention and KV cache by context length.

Context length	Attention relative to weight computation	KV cache, 8-bit
4 thousand tokens	about 7%	about 1 GB
30 thousand tokens	about 60%	about 8 GB

10.4. Gain by Environment

Table 14. Gain by execution environment and the reason.

Environment	Gain	Reason
Dedicated chip	arithmetic units 2~4 times, system several-fold up to roughly tenfold	multiplier removal and low bit width
Plain CPU	memory and bandwidth	on a CPU even multiplication is 1 SIMD clock
Machine without FPU	execution itself	integers only

On a plain CPU, table lookup (finding a value in a precomputed table) is a memory access and can be slow. Table-free paths such as the bucket inner product and SIMD addition are advantageous.

11. Conclusion

11.1. Numbers Are Two-Dimensional

Until now computers have computed by reducing numbers to one-dimensional reals. Each time, the angle was lost, and each differentiation lost the constant. The banya number keeps the number in two dimensions intact. Multiplication becomes addition, differentiation and integration become rotation, and the sum stays as an expression. There is neither conversion nor loss.

11.2. The Substance of Number Is Axiom 1 of the Banya Equation

Number is the same as Axiom 1 of the Banya equation [11].

$$\delta^2 = (\text{time} + \text{space})^2 + (\text{observer} + \text{superposition})^2 \tag{11}$$

Axiom 1 consists of two mutually orthogonal components and their magnitude δ. The magnitude and angle of the banya number are exactly this structure. That is, a number is not a value but a topology. The Banya equation shown by Axiom 1 is the substance of number, and the banya number is that substance carried over into a computer data type.

11.3. Extension: Three-Dimensional Quaternions

The present paper treats two-dimensional numbers only. The next extension is three dimensions: holding the expression intact in the form of quaternions, which handle three-dimensional rotation. Just as the two-dimensional banya number holds one angle, the quaternion banya number holds a rotation axis together with an angle. Differentiation and fractional differentiation then become rotations about an arbitrary axis.

11.4. Next Step

One banya number fits in a 32-bit union, and one AVX2 instruction performs eight banya multiplications. With this, an LLM runs with integers only on a machine without a GPU. The next step is to port this data type into an engine and show the numbers.

Paper 2 of the series runs and demonstrates an LLM with this data type ported into it, and verifies every test item of the present paper by measurement. The items to be verified are the following.

1. Restoration by the inverse operations of banya multiplication, division, differentiation, integration, and fractional differentiation, and the multiplications per instruction under SIMD loading (Sections 3 and 4, Tables 2 and 4).
2. Exactness of equality comparison on the coefficient array, early termination of magnitude comparison, and the upper bound of the normalization error (Section 5, Table 6).

3. Whether the bucket inner product agrees with the real inner product, with zero multiplications at length 4096 (Eq. 6).
4. Shift accumulation and CLZ conversion cost of the mixed structure, and the range and step size of the integer logarithm (Section 6, Table 7).
5. Whether the top-k order comparison agrees with the softmax probability order, and the subtraction division of softmax (Section 7).
6. The proportions of the three intervals of the bounded activation function, the true saturation values, and the angle clamp (Section 8, Table 10).
7. CPU token generation speed of a 7B-class model, the share of attention, and the gain by environment (Section 10, Tables 13 and 14).

Each item uses the tables and equations of the present paper directly as test items, and the measured values are recorded beside the estimates of the present paper.

1. Napier, J. *Mirifici Logarithmorum Canonis Descriptio*; Andrew Hart: Edinburgh, UK, 1614.
2. Mitchell, J.N. Computer Multiplication and Division Using Binary Logarithms. *IRE Trans. Electron. Comput.* **1962**, *EC-11*, 512-517.
3. Clenshaw, C.W.; Olver, F.W.J. Beyond Floating Point. *J. ACM* **1984**, *31*, 319-328.
4. Horowitz, M. Computing's Energy Problem (and what we can do about it). In Proceedings of the IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 9-13 February 2014; pp. 10-14.
5. Zhao, J.; Dai, S.; Venkatesan, R.; Zimmer, B.; Ali, M.; Liu, M.-Y.; Khailany, B.; Dally, W.J.; Anandkumar, A. LNS-Madam: Low-Precision Training in Logarithmic Number System Using Multiplicative Weight Update. *IEEE Trans. Comput.* **2022**, *71*, 3179-3190.
6. Sanyal, A.; Beerel, P.A.; Chugg, K.M. Neural Network Training with Approximate Logarithmic Computations. In Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Barcelona, Spain, 4-8 May 2020; pp. 3122-3126.
7. Haghi, P.; Wu, C.; Azad, Z.; Li, Y.; Gui, A.; Hao, Y.; Li, A.; et al. Bridging the Gap Between LLMs and LNS with Dynamic Data Format and Architecture Codesign. In Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO), Austin, TX, USA, 2-6 November 2024; pp. 1617-1631.
8. Hamad, H.; Qiu, Y.; Beerel, P.A.; Chugg, K.M. Bitwidth-Specific Logarithmic Arithmetic for Future Hardware-Accelerated Training. arXiv:2510.17058, 2025.
9. Tensordyne. Tensordyne Announces Breakthrough Inference System to End AI's Speed vs. Cost Trade-Off (Napier, CEO Marc Bolitho). Company announcement, 2025. Available online: https://www.tensordyne.ai/ (accessed on 30 October 2026).
10. Aussie AI. Zero-Multiplication Models, Logarithmic Models. Available online: https://www.aussieai.com/ (accessed on 30 October 2026).
11. Han, H. Banya Framework: An Axiom-Based Science Mining Engine for Deriving Fundamental Physical Constants. Zenodo **2026**. Available online: https://doi.org/10.5281/zenodo.19174651 (accessed on 30 October 2026). Axiom text: https://ubmscoin.github.io/banya
12. Gerganov, G.; et al. llama.cpp: LLM inference in C/C++. GitHub repository, 2023. Available online: https://github.com/ggml-org/llama.cpp (accessed on 30 October 2026).
13. Ma, S.; Wang, H.; Ma, L.; Wang, L.; Wang, W.; Huang, S.; Dong, L.; Wang, R.; Xue, J.; Wei, F. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764, 2024.
14. Wei, J.; Cao, S.; Cao, T.; Ma, L.; Wang, L.; Zhang, Y.; Yang, M. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. arXiv:2407.00088, 2024.